CSC 251 PROJECT 1

MORTGAGE CALCULATOR

THE ASSIGNMENT

You work for (or own) a mortgage firm that offers competitive loan products for home buyers in the Raleigh area. You will write a Python program to create a typical amortization table for a fixed-rate home mortgage and provide a summary of the costs at the end of the table. You will import a dictionary provided in a module called `rate_dict.py` to determine the annual interest rate for the loan depending upon the term and down payment desired by the user. Only this dictionary module, the `table_header.py` module, and the `math` module may be imported for this project. No other packages may be used but you may use any built-in Python functions.

CALCULATED PAYMENT

To calculate the monthly mortgage payment, you'll need to prompt the user to enter the purchase price of the home, the term (number of years to pay off the loan), and their down payment percentage (entered as a percentage to the nearest 0.1 percent). Your firm will offer them a conventional fixed-rate mortgage with a 10, 15, 20, or 30-year term based upon the latest annual rates. Since this loan is "fixed", it will keep the same interest rate, hence the same monthly payment, for the duration of the loan. While you will find dozens of online loan calculators (and financial services that want your business), you'll need the formula to compute the monthly payment assuming interest is calculated at the end of every month with monthly compounding and payments for the term of the loan. One website I would suggest is: https://www.thebalancemoney.com/calculate-mortgage-315668

You should download and store the `rate_dict.py` file in the same directory as your program. This file contains a dictionary that provides the annual percentage rate (APR) for each loan product. To import it, leave off the `.py` and include the statement:

```
import rate_dict
```

You can access values in the `rates` dictionary using the syntax `rate_dict.rates[`*key*`]` where the key will be a string that your code will concatenate from the user input to match one of the 9 loan options that your firm offers. Peruse the dictionary to investigate theses different options. For example, `rate_dict.rates["10-Year Fixed, 80% or less LTV"]` will access the 5.375% rate loan value for a 10-year conventional fixed loan with at least a 20% down payment. For example, a 10% down payment leaves you with a loan of 90% of the loan-to-value ratio or LTV. For simplicity's sake, we will not add closing costs of the loan, escrow amounts for insurance or property taxes, or points paid on the loan. We will, however, include PMI or Private Mortgage Insurance, which is an amount added to the monthly payment until after the loan-to-value ratio is less than 80%. The PMI is an additional amount paid each month but will not be added to the principal of the loan. Your code should differentiate between "monthly payment" and "payment with PMI". More on the PMI calculation later.

You might use the following values to determine if your program is correctly calculating the monthly payment and loading the dictionary.

Purchase price	Rate (%)	Term (years)	Down Payment %	Monthly Payment
$200,000	5.990	30	0.0	$ 1,197.82
250,000	5.625	20	0.0	1,737.42
380,000	4.750	15	15.0	2,512.40
510,000	4.375	10	25.0	3,941.16

Your loan amount, loan type, annual interest rate, and calculated monthly payment should be output prior to the output of the table as illustrated in the sample output. Use banker's rounding rules to round all monetary values to cents.

AMORTIZATION TABLE

As a mortgage lender, you are required to provide each client with an amortization table. This table shows a month-by-month breakdown of how much of their monthly payment pays towards interest on the outstanding principal balance and how much pays down the principal balance. As the balance is paid down, less of the monthly payment goes toward interest and more goes toward the principal.

Your table should have a title and contain 5 labeled columns, with columns 2-5 formatted to show dollars and cents. For your convenience, the header for the table is provided for you in the `table_header.py` file. You may import and use this file as a module in your code. Your table should be correctly formatted with decimal-aligned columns. The columns will contain data as follows:

1: Payment number: Starts at payment 0 to show the initial balance of the mortgage (purchase price minus downpayment), then shows the breakdown of interest and principal beginning with payment 1. It should include a line for each payment for the term of the loan.

2: Interest payment: This column shows how much of the monthly payment goes towards interest. It is calculated by multiplying one-twelfth of the annual interest rate times the current principal balance.

3: Principal payment: This column shows how much of the monthly payment goes toward reducing the principal. It is the difference between the payment amount and interest payment amount.

4: Principal balance: This column shows the amount of outstanding principal still to be paid back on the loan. Each month the principal balance from the previous month is reduced by the principal payment. You will notice that the principal balance decreases slowly at first but is reduced rapidly as the term of the loan progresses.

5. Primary Mortgage Insurance (PMI) payment: This additional monthly charge is required by most financial institutions **until the month after** the LTV of the loan falls below 80%, after which the PMI payment is $0. To read more, go to: https://www.nerdwallet.com/article/mortgages/pmi-private-mortgage-insurance

The PMI is based on the initial LTV ratio and the credit score of the borrower. Let's assume your borrower has good credit so will only consider their LTV which is dependent upon their down payment according to the following chart:

LTV Range of Initial Loan	Annual PMI Rate
95% < LTV <= 100%	1.040%
90% < LTV <= 95%	0.875%
85% < LTV <= 90%	0.685%
80% < LTV <= 85%	0.475%
LTV <= 80%	0.000%

The annual PMI rate is multiplied times the loan amount and is spread evenly over the 12 monthly payments. PMI is calculated at the beginning of the loan and remains the same monthly amount until it is dropped. The PMI payments don't change from month-to-month and they have no impact on the principal balance. Once the outstanding principal balance falls below 80% of the purchase price, PMI payments are eliminated **beginning the following month**.

Note: Since the purpose of an amortization table is to display the outstanding balance left on your loan if you pay back the money according to schedule, you would expect the outstanding balance to be exactly $0.00 after the last payment. Since the payment amounts are rounded to the nearest cent each month (in addition to the floating point error), the final payment must often be adjusted up or down by a few dollars to account for this rounding. For your table, assume the same payment for all months, so your final balance may be off by a few cents or a few dollars. Don't let this bother you! It does not need to be forced to $0.00. The actual final payment will be adjusted such that the total principal paid is equal to the loan amount.

MORTGAGE SUMMARY

After the table, summarize the total amounts paid in interest, on principal, in PMI (if any), and the total of all payments. These values should be aligned with the 5th column of the table. (This might not be obvious with the page break in the sample output below.)

SAMPLE INPUT AND OUTPUT

The following shows sample output for a 10-Year Fixed, 90.1-100% LTV mortgage on a $200,000 home with a 5% down payment. The values in **bold** indicate user input and may not be included in your output file unless.

```
Enter the purchase price of your home: $200000
Enter the term of the loan in years: 10
Enter your down payment percentage: 5.0

Loan amount: $190,000.00
Loan type: 10-Year Fixed, 90.1-100% LTV at 4.625% APR
Monthly Payment = $1,980.60
                          Amortization Table

Payment    Interest Payment    Principal Payment     Principal Balance       PMI
   #             ($)                  ($)                   ($)              ($)
================================================================================
   0                                                    190,000.00
   1            732.29              1,248.31           188,751.69           138.54
   2            727.48              1,253.12           187,498.57           138.54
   3            722.65              1,257.95           186,240.62           138.54
   4            717.80              1,262.80           184,977.82           138.54
   5            712.94              1,267.66           183,710.16           138.54
   6            708.05              1,272.55           182,437.61           138.54
   7            703.14              1,277.46           181,160.15           138.54
   8            698.22              1,282.38           179,877.77           138.54
   9            693.28              1,287.32           178,590.45           138.54
  10            688.32              1,292.28           177,298.17           138.54
  11            683.34              1,297.26           176,000.91           138.54
  12            678.34              1,302.26           174,698.65           138.54
  13            673.32              1,307.28           173,391.37           138.54
  14            668.28              1,312.32           172,079.05           138.54
  15            663.22              1,317.38           170,761.67           138.54
```

16	658.14	1,322.46	169,439.21	138.54
17	653.05	1,327.55	168,111.66	138.54
18	647.93	1,332.67	166,778.99	138.54
19	642.79	1,337.81	165,441.18	138.54
20	637.64	1,342.96	164,098.22	138.54
21	632.46	1,348.14	162,750.08	138.54
22	627.27	1,353.33	161,396.75	138.54
23	622.05	1,358.55	160,038.20	138.54
24	616.81	1,363.79	158,674.41	138.54
25	611.56	1,369.04	157,305.37	0.00
26	606.28	1,374.32	155,931.05	0.00
27	600.98	1,379.62	154,551.43	0.00
28	595.67	1,384.93	153,166.50	0.00
29	590.33	1,390.27	151,776.23	0.00
30	584.97	1,395.63	150,380.60	0.00
31	579.59	1,401.01	148,979.59	0.00
32	574.19	1,406.41	147,573.18	0.00
33	568.77	1,411.83	146,161.35	0.00
34	563.33	1,417.27	144,744.08	0.00
35	557.87	1,422.73	143,321.35	0.00
36	552.38	1,428.22	141,893.13	0.00
37	546.88	1,433.72	140,459.41	0.00
38	541.35	1,439.25	139,020.16	0.00
39	535.81	1,444.79	137,575.37	0.00
40	530.24	1,450.36	136,125.01	0.00
41	524.65	1,455.95	134,669.06	0.00
42	519.04	1,461.56	133,207.50	0.00
43	513.40	1,467.20	131,740.30	0.00
44	507.75	1,472.85	130,267.45	0.00
45	502.07	1,478.53	128,788.92	0.00
46	496.37	1,484.23	127,304.69	0.00
47	490.65	1,489.95	125,814.74	0.00
48	484.91	1,495.69	124,319.05	0.00
49	479.15	1,501.45	122,817.60	0.00
50	473.36	1,507.24	121,310.36	0.00
51	467.55	1,513.05	119,797.31	0.00
52	461.72	1,518.88	118,278.43	0.00
53	455.86	1,524.74	116,753.69	0.00
54	449.99	1,530.61	115,223.08	0.00
55	444.09	1,536.51	113,686.57	0.00
56	438.17	1,542.43	112,144.14	0.00
57	432.22	1,548.38	110,595.76	0.00
58	426.25	1,554.35	109,041.41	0.00
59	420.26	1,560.34	107,481.07	0.00
60	414.25	1,566.35	105,914.72	0.00
61	408.21	1,572.39	104,342.33	0.00
62	402.15	1,578.45	102,763.88	0.00
63	396.07	1,584.53	101,179.35	0.00
64	389.96	1,590.64	99,588.71	0.00
65	383.83	1,596.77	97,991.94	0.00
66	377.68	1,602.92	96,389.02	0.00
67	371.50	1,609.10	94,779.92	0.00
68	365.30	1,615.30	93,164.62	0.00
69	359.07	1,621.53	91,543.09	0.00
70	352.82	1,627.78	89,915.31	0.00
71	346.55	1,634.05	88,281.26	0.00
72	340.25	1,640.35	86,640.91	0.00
73	333.93	1,646.67	84,994.24	0.00

74	327.58	1,653.02	83,341.22	0.00
75	321.21	1,659.39	81,681.83	0.00
76	314.82	1,665.78	80,016.05	0.00
77	308.40	1,672.20	78,343.85	0.00
78	301.95	1,678.65	76,665.20	0.00
79	295.48	1,685.12	74,980.08	0.00
80	288.99	1,691.61	73,288.47	0.00
81	282.47	1,698.13	71,590.34	0.00
82	275.92	1,704.68	69,885.66	0.00
83	269.35	1,711.25	68,174.41	0.00
84	262.76	1,717.84	66,456.57	0.00
85	256.13	1,724.47	64,732.10	0.00
86	249.49	1,731.11	63,000.99	0.00
87	242.82	1,737.78	61,263.21	0.00
88	236.12	1,744.48	59,518.73	0.00
89	229.40	1,751.20	57,767.53	0.00
90	222.65	1,757.95	56,009.58	0.00
91	215.87	1,764.73	54,244.85	0.00
92	209.07	1,771.53	52,473.32	0.00
93	202.24	1,778.36	50,694.96	0.00
94	195.39	1,785.21	48,909.75	0.00
95	188.51	1,792.09	47,117.66	0.00
96	181.60	1,799.00	45,318.66	0.00
97	174.67	1,805.93	43,512.73	0.00
98	167.71	1,812.89	41,699.84	0.00
99	160.72	1,819.88	39,879.96	0.00
100	153.70	1,826.90	38,053.06	0.00
101	146.66	1,833.94	36,219.12	0.00
102	139.59	1,841.01	34,378.11	0.00
103	132.50	1,848.10	32,530.01	0.00
104	125.38	1,855.22	30,674.79	0.00
105	118.23	1,862.37	28,812.42	0.00
106	111.05	1,869.55	26,942.87	0.00
107	103.84	1,876.76	25,066.11	0.00
108	96.61	1,883.99	23,182.12	0.00
109	89.35	1,891.25	21,290.87	0.00
110	82.06	1,898.54	19,392.33	0.00
111	74.74	1,905.86	17,486.47	0.00
112	67.40	1,913.20	15,573.27	0.00
113	60.02	1,920.58	13,652.69	0.00
114	52.62	1,927.98	11,724.71	0.00
115	45.19	1,935.41	9,789.30	0.00
116	37.73	1,942.87	7,846.43	0.00
117	30.24	1,950.36	5,896.07	0.00
118	22.72	1,957.88	3,938.19	0.00
119	15.18	1,965.42	1,972.77	0.00
120	7.60	1,973.00	-0.23	0.00

Mortgage Summary

Total interest paid: $	47,671.77
Total payment to principal: $	190,000.00
Total PMI paid: $	3,324.96
Total payments: $	240,996.73

Input and expected output files are provided for you to test your program.

OTHER PROGRAM REQUIREMENTS

Your program should be adequately documented with an opening comment and docstring comments for **each** function.

Input validation and exception handling should be implemented to make your code "unbreakable."

Your program should include the following functions as well as others you choose to use to organize your code:

`make_table_header()` – This function is given in the `table_header.py` file to provide you with a guide for the formatting of the table. Use as is.

`output_loan_info(`*`str`*`)` – This function is passed a string used as the key for the rates dictionary and outputs the loan type with the rate information in the format:

```
Loan type: 10-Year Fixed, 90.1-100% LTV at 4.625% APR
```

You should not return a value from this function.

`calc_PMI(`*`float, float`*`)` – This function is passed the purchase price and down payment percentage as float values and returns the monthly PMI payment in dollars and cents.

Since this program does not use file I/O and is meant to be interactive, include input validation after EACH value to allow your user to reenter the value until they enter an appropriate value. For example, if they enter a term of 12 years, which is NOT one of the mortgage products that you offer, prompt them to reenter a term until they enter a 10, 15, 20, or 30. The test files include some inappropriate values followed by appropriate ones that your code should catch. Your input validation should immediately follow the input of each value. You should not wait until the user has entered all values then ask them to reenter any or all of them again.

You code should import and use the provided `table_header.py` and `rate_dict.py` files without making any changes to them. Submit only your `Project1LastNameFirstInitial.py` file to Moodle.